

IE MÓDULO SISTEMAS INFORMÁTICOS		
Curso/grupo: 1º DAM	Módulo	0441
Fecha:		UT 2
Entrega:		Manejo Docker

INSTALACIÓN Y MANEJO BÁSICO DE CONTENEDORES DOCKER

Instalación en GNU/Linux utilizando repositorios

<https://docs.docker.com/install/linux/docker-ce/ubuntu/>

Antes de instalar Docker por primera vez en nuestro equipo, necesitamos configurar el repositorio de docker. A continuación, ya podremos instalar y actualizar Docker desde el repositorio. *(OJO, que a veces la terminal no pilla bien el Copy/Paste).* **Si se es inexperto, se recomienda realizarlo en máquina virtual, a fin de no perjudicar el sistema anfitrión.**

Instalación en Windows

<https://www.docker.com/products/docker-desktop>

<https://docs.docker.com/docker-for-windows/install/>

Una vez instalado docker, comprobamos que está bien instalado ejecutando un "hello world"

```
$ sudo docker run hello-world
```

Este comando descarga una imagen de prueba y la ejecuta en un contenedor. Cuando el contenedor se ejecuta, imprime un mensaje informativo y sale.

Docker CE está instalado y funcionando. Se crea el grupo de ventana acoplable pero no se agrega ningún usuario.

IMPORTANTE: Necesitamos usar **sudo** para ejecutar los comandos de Docker. Para evitar la tediosa tarea de tener que estar siempre dando privilegios al usuario que estamos utilizando, podemos agregar este usuario al grupo "docker" de la siguiente manera:

```
$ sudo usermod -aG docker usuario
```

¡OJO! ¡ES NECESARIO CERRAR SESIÓN EN UBUNTU Y VOLVER A INICIAR PARA QUE ESTO TENGA EFECTO!

IE MÓDULO SISTEMAS INFORMÁTICOS		
Curso/grupo: 1º DAM	Módulo	0441
Fecha:		UT 2
Entrega:		Manejo Docker

MANEJO DE DOCKER

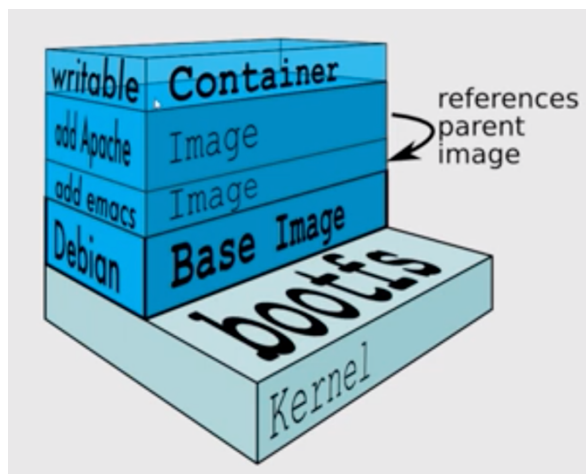
Diferencia entre Imágenes y contenedores

Imágenes:

- Plantilla de solo lectura vacía o con una aplicación preinstalada para la creación de contenedores
- Creadas por nosotros o por la comunidad (Docker Hub) (Docker Search)

Contenedores:

- Contiene lo necesario para ejecutar nuestras aplicaciones
- Basados en una o más imágenes.



Primeros comandos

Los primeros comandos que tenemos que empezar a utilizar son los que nos dan información sobre nuestro sistema:

El comando *docker* nos va a mostrar todas las opciones y posibilidad que podemos escribir y utilizar.

```
$ docker
```

Con *docker version* podremos comprobar la versión del cliente y del servidor que tenemos instalada: versión instalada, versión del Lenguaje de programación *Go* (de Google con el que está desarrollado Docker), etc.

```
$ docker version
```

Con el comando *docker info* podemos visualizar toda la información de la instalación de nuestro docker: nº contenedores y su estado (ejecutándose, pausados o parados), imágenes descargadas, versión del servidor, plugin, versión del SO y del kernel, etc.

```
$ docker info
```

IE MÓDULO SISTEMAS INFORMÁTICOS		
Curso/grupo: 1º DAM	Módulo	0441
Fecha:		UT 2
Entrega:		Manejo Docker

Si queremos obtener más información sobre algún comando que estemos utilizando tan solo tenemos que poner:

```
$ docker comando --help
```

Ahí nos aparecerá una breve descripción de cuál es el cometido del comando y las opciones disponibles que puedes agregarle.

Docker hub: En *docker hub* tenemos disponibles cientos de imágenes ya configuradas con aplicaciones que podemos utilizar de forma libre ya que son publicadas por la comunidad.

<https://hub.docker.com> Nos podemos registrar para subir nuestras imágenes o directamente nos vamos a la barra de búsqueda. En la actualidad tenemos disponibles más de 2 millones de imágenes.

Existen dos tipos de imágenes: las oficiales de la compañía o comunidad que desarrolla dicha tecnología y las que individualmente crean y suben usuarios. Típico ejemplo de ello es Ubuntu. La primera entrada de la búsqueda que hacemos en docker hub nos aparece la imagen oficial que la empresa/comunidad Canonical ha realizado.

También se puede visualizar este listado de imágenes por el terminal ejecutando el comando *docker search* seguido de la imagen que estás buscando

```
$ docker search imagen
```

Nos aparecerá una relación de todas las imágenes disponibles con el nombre, descripción, stars (recomendaciones), etc. **Por norma general, las imágenes están prediseñadas para una utilidad concreta, habiendo por ello multitud de tipos de imágenes según para el cometido que se requiera.**

Se puede hacer búsquedas teniendo en cuenta el TAG (versión de una imagen de docker) por ejemplo *latest* es la imagen base o estándar. Para ello utilizamos los dos puntos y especificamos según el tag que queramos filtrar:

```
$ docker search ubuntu:18.04
```

Para descargar una imagen concreta, hacemos uso del comando *pull*

```
$ docker pull imagen_descargar
```

Para saber exactamente como escribir el comando pull para descargar la imagen, basta con pinchar en docker hub sobre la imagen que estamos interesados y la derecha nos aparecerá cómo debemos teclearla en la terminal:

Copy and paste to pull this image

```
docker pull ubuntu
```



[View Available Tags](#)

Realizado por el profesor D. Andrés Guillén Jiménez

IE MÓDULO SISTEMAS INFORMÁTICOS		
Curso/grupo: 1º DAM	Módulo	0441
Fecha:		UT 2
Entrega:		Manejo Docker

El proceso se compone en varias descargas que puede durar varios minutos.

¿Cómo podemos ver nuestra imagen descargada y las demás que tenemos descargadas anteriormente? Pues con la opción *images*

```
$ docker images
```

Donde aparte del nombre, podemos ver el TAG, imageID (id que se invoca), etc.

Para conocer los contenedores que están en ejecución:

```
$ docker ps
```

Con la opción *-a* podemos visualizar los contenedores que se han creado, se han ejecutado, etc. porque nos aparece su situación en la columna *STATUS*

```
$ docker ps -a
```

El comando *run* es el encargado de ejecutar contenedores a partir de imágenes.

Por ejemplo:

```
$ docker run ubuntu ls
```

Estamos diciendo a docker que ejecute un contenedor con Ubuntu y que nos liste el contenido de la raíz de este SO. Si a continuación de esto escribimos un *ps -a*, veremos que el status de ese contenedor es *exited*.

IMPORTANTE: Los contenedores solo aparecerán en ejecución mientras estén ejecutando algún comando o proceso. Finalizado ésto, pasarán al estatus *Exited*, indicando que el contenedor ya ha salido de todas las ejecuciones que tenía pendiente.

¿Qué ocurre si yo quiero realizar operaciones y poder trabajar con ese contenedor interactuando con él? Pongamos el ejemplo de que queremos abrir una terminal en Ubuntu para poder realizar acciones en él. Para eso escribiríamos las siguientes opciones (*-i -t* ó *-it*) para que sea interactivo:

```
$ docker run -i -t ubuntu bash
```

Lo primero que advertimos es que el prompt nos ha cambiado. Ahora nos aparece el prompt del contenedor ubuntu. Por ejemplo: *root@a61bca7963cf* el cual nos está indicando que el usuario que está trabajando es el *root* y el *ID* del contenedor que se está ejecutando. Desde aquí ya podemos trabajar con este contenedor.

Si ahora abrimos otra terminal dejando ésta al lado y ejecutamos un *ps -a* visualizamos que este contenedor está en ejecución y el tiempo que lleva ejecutándose (Up 50s).

Realizado por el profesor D. Andrés Guillén Jiménez

IE MÓDULO SISTEMAS INFORMÁTICOS		
Curso/grupo: 1º DAM	Módulo	0441
Fecha:		UT 2
Entrega:		Manejo Docker

Para salir basta con escribir en la terminal *exit* y volvemos a nuestra anterior sesión. Para borrar contenedores, basta con poner el comando de borrar seguido de los dos primeros dígitos de su ID (el identificador está en código hexadecimal).

```
$ docker rm 01
```

Podemos crear un contenedor con un nombre, para así poder identificarlo mejor:

```
$ docker run --name NOMBRE -it ubuntu
```

Esto puede ser útil cuando creamos un contenedor con alguna característica y queremos que sea fácil de reconocer.

Otra particularidad importante es si queremos salir del contenedor interactivo sin que se finalice. Para ello pulsamos: *Control+p+q*. Si ahora vemos todos los procesos con el comando *ps* observamos que el contenedor sigue ejecutándose.

Para volver a conectarnos a este contenedor del cuál nos hemos salido sin finalizar:

```
$ docker attach id
```

Donde el id con tan solo 2 dígitos suele detectar de qué contenedor nos estamos refiriendo.

El contenedor volverá a ejecutarse con el comando que tenía lanzado (como por ejemplo el contenedor anterior con *bash*). El id siempre será el mismo, aunque nos salgamos y entremos varias veces y la información que se cree y esté dentro también se conservará.

Con los comandos *stop* y *start* detenemos y lo volvemos a iniciar:

```
$ docker stop id
```

```
$ docker start id
```

Una vez realizado el *start*, para trabajar con él utilizamos el comando anterior *attach*. Uno de los usos que tiene el *stop* es que si nosotros nos salimos del contenedor (*Control+p+q*) pero éste sigue ejecutándose, no se puede eliminar. O tenemos que volver hacia él o ejecutamos *stop*. Además, también nos permite iniciar un contenedor del cual ya nos hayamos ejecutando *start* para iniciarlo y *attach* para conectarnos. Se mantienen los archivos y configuración que ya hubiéramos creado.

IE MÓDULO SISTEMAS INFORMÁTICOS		
Curso/grupo: 1º DAM	Módulo	0441
Fecha:		UT 2
Entrega:		Manejo Docker

PERSONALIZAR CONTENEDORES

Ya sabemos crear, ejecutar, parar y volver a iniciar contenedores. Ahora vamos a ver cómo podemos personalizar éstos instalando paquetes, IDE, servidor Web, Git, apache, MySQL, PHP, Python, etc. Lo que veamos conveniente. Eso si, una vez arrancado el contenedor y antes de instalar nada, lo primero es actualizar el repositorio con *update*. Una vez hecho ya iremos instalando y configurando según nuestras necesidades.

Podemos crear una imagen a partir de un contenedor ya personalizado a través del comando *commit*:

```
$ docker run -it ubuntu /bin/bash
```

```

andres@andres-Parallels-Virtual-Platform: ~
Archivo  Editar  Ver  Buscar  Terminal  Ayuda

root@8743b382b1a7:/# ls
bin  dev  home  lib64  mnt  proc  run  srv  tmp  var
boot  etc  lib  media  opt  root  sbin  sys  usr
root@8743b382b1a7:/# mkdir prueba
root@8743b382b1a7:/# cd prueba
root@8743b382b1a7:/prueba# touch msgPrueba
root@8743b382b1a7:/prueba# cd ..
root@8743b382b1a7:/# ls
bin  dev  home  lib64  mnt  proc  root  sbin  sys  usr
boot  etc  lib  media  opt  prueba  run  srv  tmp  var
root@8743b382b1a7:/# read escape sequence

```

Nos salimos del contenedor y ejecutamos:

```

andres@andres-Parallels-Virtual-Platform:~$ docker ps -a
CONTAINER ID   PORTS   IMAGE      COMMAND                  CREATED        STATUS
8743b382b1a7   inutes   ubuntu     "/bin/bash"             About an hour ago   Up 12 m
silly_khorana

andres@andres-Parallels-Virtual-Platform:~$ docker commit 87 ubuntu_prueba
sha256:7234eb2bfab8e4054c1df5f428ce4b24f6c563b98ad875eb4be0436a67532187

andres@andres-Parallels-Virtual-Platform:~$ docker ps -a
CONTAINER ID   PORTS   IMAGE      COMMAND                  CREATED        STATUS
8743b382b1a7   inutes   ubuntu     "/bin/bash"             About an hour ago   Up 12 m
silly_khorana

andres@andres-Parallels-Virtual-Platform:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
ubuntu_prueba  latest   7234eb2bfab8   18 seconds ago   88.9MB
<none>        <none>   1fc7ce31ee9a   13 minutes ago   88.9MB
ubuntu        18.04    94e814e2efa8   4 weeks ago     88.9MB
ubuntu        latest   94e814e2efa8   4 weeks ago     88.9MB
hello-world    latest   fce289e99eb9   3 months ago    1.84kB

```

```
$ docker commit id NOMBRE
```

donde el *id* es el identificador del contenedor y *NOMBRE* el nombre que le queremos poner.

Realizado por el profesor D. Andrés Guillén Jiménez

IE MÓDULO SISTEMAS INFORMÁTICOS		
Curso/grupo: 1º DAM	Módulo	0441
Fecha:		UT 2
Entrega:		Manejo Docker

Otra opción para realizar el commit es la siguiente:

```
$ docker commit -m "comentario" -a "autor" id_container
NOMBRE:target
```

Por ejemplo:

```
$ docker commit -m "comentario de los cambios" -a "Andres Guillen" 87
ubuntu_prueba:latest
```

Con esa nueva imagen que hemos generado podemos crear un contenedor (podemos comprobar que es completamente diferente al anterior visualizando sus ID) y comprobar que el contenido que se creó está presente. A partir de aquí podemos trabajar e instalar lo que queramos.

Esta acción de crear una imagen a través de un contenedor ya personalizado tiene la utilidad de que si queremos dejar ya preinstalado un par de herramientas o configuraciones que en el futuro nos harán falta en varias o múltiples ocasiones. Esto, nos ahorrará mucho tiempo tedioso de ir instalando y configurando cada vez que creemos un contenedor.

Supongamos por ejemplo que queremos a partir de este contenedor instalarle un servidor Web apache.

```
root@52dcf75db26c: /
Archivo Editar Ver Buscar Terminal Ayuda
root@52dcf75db26c:/# apt-get update
Hit:1 http://archive.ubuntu.com/ubuntu bionic InRelease
Hit:2 http://security.ubuntu.com/ubuntu bionic-security InRelease
Hit:3 http://archive.ubuntu.com/ubuntu bionic-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu bionic-backports InRelease
Reading package lists... Done
root@52dcf75db26c:/# apt-get install apache2
```

Una vez finalizado comprobamos y arrancamos el servicio del servidor apache

```
root@52dcf75db26c:/# service apache2 status
* apache2 is not running
root@52dcf75db26c:/# service apache2 start
* Starting Apache httpd web server apache2
AH00558: apache2: Could not reliably determine the server's fully qualified domain name
, using 172.17.0.2. Set the 'ServerName' directive globally to suppress this message
*
root@52dcf75db26c:/# service apache2 status
* apache2 is running
```

Ahora vamos a hacer una imagen con *commit* pero de diferente forma. Le vamos a pasar varios parámetros que vamos a modificar dentro de ese contenedor. Para eso nos salimos con Control+q+p y escribimos el siguiente commit:

```
$ docker commit --change='CMD ["apachectl","-DFOREGROUND"]' -c
"EXPOSE 85" 52dcf75db26c apache2
```


IE MÓDULO SISTEMAS INFORMÁTICOS		
Curso/grupo: 1º DAM	Módulo	0441
Fecha:		UT 2
Entrega:		Manejo Docker

```
docker commit --change='CMD ["apachectl","-DFOREGROUND"]' -c "EXPOSE 85" idcontenedor apache2
```

```
andres@andres-Parallels-Virtual-Platform:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
207b08a03860   ubuntu_prueba  "/bin/bash"             41 minutes ago Up 13 minutes                eager_antonelli
andres@andres-Parallels-Virtual-Platform:~$ docker commit --change='CMD ["apachectl","-DFOREGROUND"]' -c "EXPOSE 85" 207b08a03860 apache2
sha256:23567b8859cff334bc2fcd0e77ce8147f29f4ab59d0097a61f8e90e4dd34808a
andres@andres-Parallels-Virtual-Platform:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
apache2       latest   23567b8859cf   13 seconds ago  227MB
```

Vemos que cuando se nos crea la imagen aparece a la izquierda *sha256*. Esa es la diferencia al crear una imagen y un contenedor, puesto que al crear el contenedor no aparece.

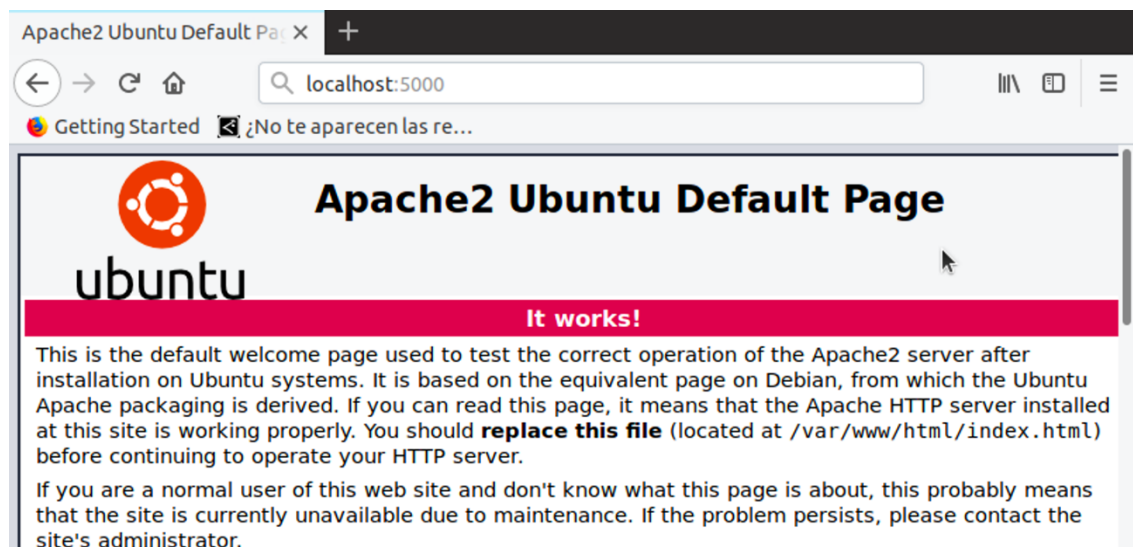
Creamos ahora por ejemplo un contenedor con esta imagen Apache, pasándoles las banderas `-d` (para lanzarlo en segundo plano) `-p` (para especificar el puerto) 5000 con el que nos vamos a conectar a través del puerto 80 (puerto HTTP) y el nombre de la imagen que vamos a utilizar

```
$ docker run -d -p 5000:80 apache2
```

Aunque cuando definimos la imagen le definimos el Puerto 85, ahora al crear el contenedor le hemos definido que nos conectemos por el puerto 5000.

```
andres@andres-Parallels-Virtual-Platform:~$ docker run -d -p 5000:80 apache2
23467add7a570303dc12b9df13e5315f3f166f26dbe3f4dab34698867392a370
andres@andres-Parallels-Virtual-Platform:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
23467add7a57   apache2    "apachectl -DFOREGRO..." 7 seconds ago Up 6 seconds    85/tcp, 0.0.0.0:5000->80/tcp   angry_dubinsky
207b08a03860   ubuntu_prueba  "/bin/bash"             43 minutes ago Up 15 minutes                eager_antonelli
andres@andres-Parallels-Virtual-Platform:~$
```

Si nos vamos al navegador y escribimos *localhost:5000* nos aparecerá la pantalla de bienvenida de Apache indicándonos que el servidor está listo para trabajar con él.



En el siguiente enlace podemos encontrar todos los comandos con su descripción, usos y opciones.

<https://docs.docker.com/v17.09/engine/reference/run/>

Realizado por el profesor D. Andrés Guillén Jiménez

IE MÓDULO SISTEMAS INFORMÁTICOS		
Curso/grupo: 1º DAM	Módulo	0441
Fecha:		UT 2
Entrega:		Manejo Docker

DockerFile

Para realizar imágenes a partir de un contenedor personalizado o de un contenedor básico de manera **más sencilla** podemos utilizar Dockerfile.

Dockerfile es un documento de texto que contiene todos los comandos que un usuario tiene que ejecutar usando la línea de comandos, para, montar una imagen. Para hacerlo más entendible podríamos comparar un dockerfile con “una receta de cocina de una comida”.

Distintas Partes (o ingredientes) de un dockerfile que podemos crear:

FROM: Definir una imagen base para crear nuestra nueva imagen con Dockerfile (puede ser una imagen ya descargada en nuestro Pc o de docker hub)

FROM ubuntu 18.04

MAINTAINER: Hace referencia al creador de la receta (también para definir el mantenimiento).

MAINTAINER Andres andres.guillen@ejemplo.org

Introducimos esta orden para que no nos pregunte por la región donde va a estar el servidor

ARG DEBIAN_FRONTEND=noninteractive

RUN: Nos permite ejecutar comando en la imagen antes de ser creada

RUN apt-get update && apt-get install apache

ADD/COPY: Nos permite agregar o copiar archivos desde el equipo local a la imagen

ADD index.html /var/www/html (en esta ubicación es donde se encuentra apache por defecto)

EXPOSE: Nos permite exponer por defecto un puerto para el contenedor

EXPOSE 8080

CMD: Ejecutar acción por defecto al crear el contenedor, es la finalidad, donde el servicio es apache2ctl para poder inicializarlo

CMD [“/usr/sbin/apache2ctl”, “-D”, “BACKGROUND”]

Hagamos el siguiente ejemplo:

Creamos una carpeta en nuestra carpeta personal llamada *miweb*. Dentro de ella llamamos a un editor de texto, por ejemplo, *gedit* y escribimos lo siguiente (teniendo en cuenta lo descrito anteriormente):

FROM ubuntu:18.04

MAINTAINER Andres andres.guillen@ejemplo.org

ARG DEBIAN_FRONTEND=noninteractive

RUN apt-get update

RUN apt-get install -y apache2

EXPOSE 81

CMD /usr/sbin/apache2ctl -D FOREGROUND

IE MÓDULO SISTEMAS INFORMÁTICOS		
Curso/grupo: 1º DAM	Módulo	0441
Fecha:		UT 2
Entrega:		Manejo Docker

Cuando creamos nuestro archivo y lo vamos a guardar con toda la información rellena debemos nombrarlo OBLIGATORIAMENTE como “Dockerfile”.

Para confirmarlo listamos el contenido del directorio y mostramos el contenido del fichero utilizando *ls* y *cat Dockerfile* respectivamente.

```
andres@andres-Parallels-Virtual-Platform:~/miweb$ ls
Dockerfile
andres@andres-Parallels-Virtual-Platform:~/miweb$ cat Dockerfile
FROM ubuntu:18.04
MAINTAINER Andres andres.guillen@ejemplo.org
RUN apt-get update
RUN apt-get -y install apache2
EXPOSE 81
CMD /usr/sbin/apache2ctl -D FOREGROUND

andres@andres-Parallels-Virtual-Platform:~/miweb$
```

Ahora vamos a crear nuestra imagen a partir de este Dockerfile. Para esto vamos a utilizar el comando *docker build*.

```
$ docker build -t nombre_imagen rutaAbsoluta
```

donde el *nombre_imagen* es el nombre de la imagen que vamos a generar y *rutaAbsoluta* es la ruta completa donde se encuentra la carpeta en la que hemos almacenado el Dockerfile.

```
andres@andres-Parallels-Virtual-Platform:~/miweb$ docker build -t miweb /home/an
dres/miweb/
Sending build context to Docker daemon  2.048kB
Step 1/6 : FROM ubuntu:18.04
--> 94e814e2efa8
Step 2/6 : MAINTAINER Andres andres.guillen@ejemplo.org
--> Running in c5deacad2435
Removing intermediate container c5deacad2435
--> 825dd0f02fd4
Step 3/6 : RUN apt-get update
--> Running in 280882fe6b04
Get:1 http://archive.ubuntu.com/ubuntu bionic InRelease [242 kB]
Get:2 http://security.ubuntu.com/ubuntu bionic-security InRelease [88.7 kB]
Get:3 http://security.ubuntu.com/ubuntu bionic-security/main amd64 Packages [21
16 kB]
Get:4 http://archive.ubuntu.com/ubuntu bionic-updates InRelease [88.7 kB]
Get:5 http://archive.ubuntu.com/ubuntu bionic-backports InRelease [74.6 kB]
Get:6 http://archive.ubuntu.com/ubuntu bionic/universe amd64 Packages [11.3 MB]
Get:7 http://security.ubuntu.com/ubuntu bionic-security/multiverse amd64 Package
s [24.7 kB]
Get:8 http://security.ubuntu.com/ubuntu bionic-security/restricted amd64 Package
s [396 kB]
Get:9 http://security.ubuntu.com/ubuntu bionic-security/universe amd64 Packages
[1410 kB]
```

En cada *Step* (Step 1/6, Step 2/6, etc.) va realizando secuencialmente los pasos que le hemos definido.

Una vez acabado nos debe aparecer al final de todas las líneas de instalación que la construcción (built) se ha creado bien.

IE MÓDULO SISTEMAS INFORMÁTICOS		
Curso/grupo: 1º DAM	Módulo	0441
Fecha:		UT 2
Entrega:		Manejo Docker

```
Successfully built 09bca04fe612
Successfully tagged miweb:latest
andres@andres-Parallels-Virtual-Platform:~/miweb$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
miweb	latest	09bca04fe612	6 minutes ago	210MB
ubuntu_prueba	latest	c4cfe4e4cda1	2 hours ago	88.9MB
apache_2	latest	6b4d2b224886	4 weeks ago	210MB
<none>	<none>	832c4be243c1	4 weeks ago	210MB
apache2	latest	36e9e2a69a01	4 weeks ago	210MB
<none>	<none>	1fc7ce31ee9a	4 weeks ago	88.9MB
ubuntu	18.04	94e814e2efa8	2 months ago	88.9MB
ubuntu	latest	94e814e2efa8	2 months ago	88.9MB
hello-world	latest	fce289e99eb9	4 months ago	1.84kB

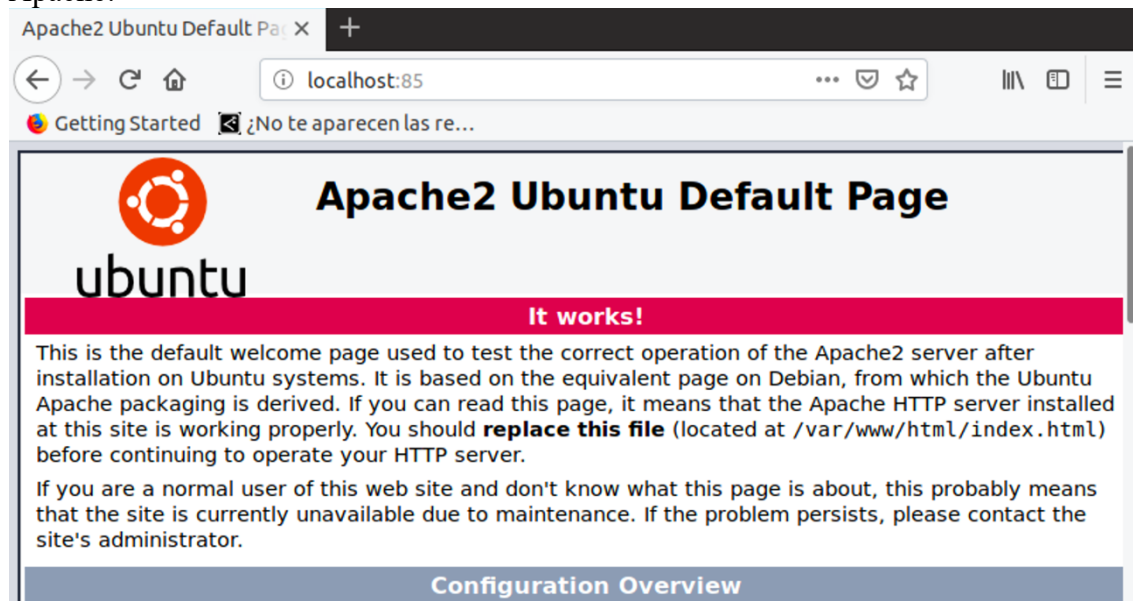
Si mostramos las imágenes, veremos que nos aparece la que acabamos de crear.

Si creamos ahora un contenedor especificando puertos nos quedaría así:

```
andres@andres-Parallels-Virtual-Platform:~/miweb$ docker run -d -p 85:80 miweb
80dcb3bd89818767d0b3449aac5f786f57e1a6c8a201e2c8a34a3db6d2571d5c
andres@andres-Parallels-Virtual-Platform:~/miweb$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	ST
ATUS	PORTS	NAMES		
80dcb3bd8981	miweb	"/bin/sh -c '/usr/sb..."	22 seconds ago	Up
21 seconds	81/tcp, 0.0.0.0:85->80/tcp	xenodochial_ride		

Ejecutando *docker ps* vemos que efectivamente este contenedor se está ejecutando y su puerto para http es el 85. Si abrimos el navegador y escribimos como antes el localhost, pero ahora indicando que sea por el puerto 85 nos aparecerá la pagina de inicio de Apache:



¡¡VEAMOS OTRO EJEMPLO!!

Aquí nos vamos a descargar a través de github una web, que luego descomprimiremos y copiaremos a la ruta por defecto de apache2

FROM ubuntu:18.04

MAINTAINER Andres andres.guillen@ejemplo.org

ARG DEBIAN_FRONTEND=noninteractive

RUN apt-get update

RUN apt-get install -y apache2

RUN apt-get install -y wget

Realizado por el profesor D. Andrés Guillén Jiménez

IE MÓDULO SISTEMAS INFORMÁTICOS		
Curso/grupo: 1º DAM	Módulo	0441
Fecha:		UT 2
Entrega:		Manejo Docker

RUN apt-get install -y unzip

RUN wget https://github.com/BlackrockDigital/startbootstrap-freelancer/archive/gh-pages.zip

RUN unzip gh-pages.zip

RUN cp -a /startbootstrap-freelancer-gh-pabges/ var/www/html*

EXPOSE 82

CMD /usr/sbin/apache2ctl -D FOREGROUND

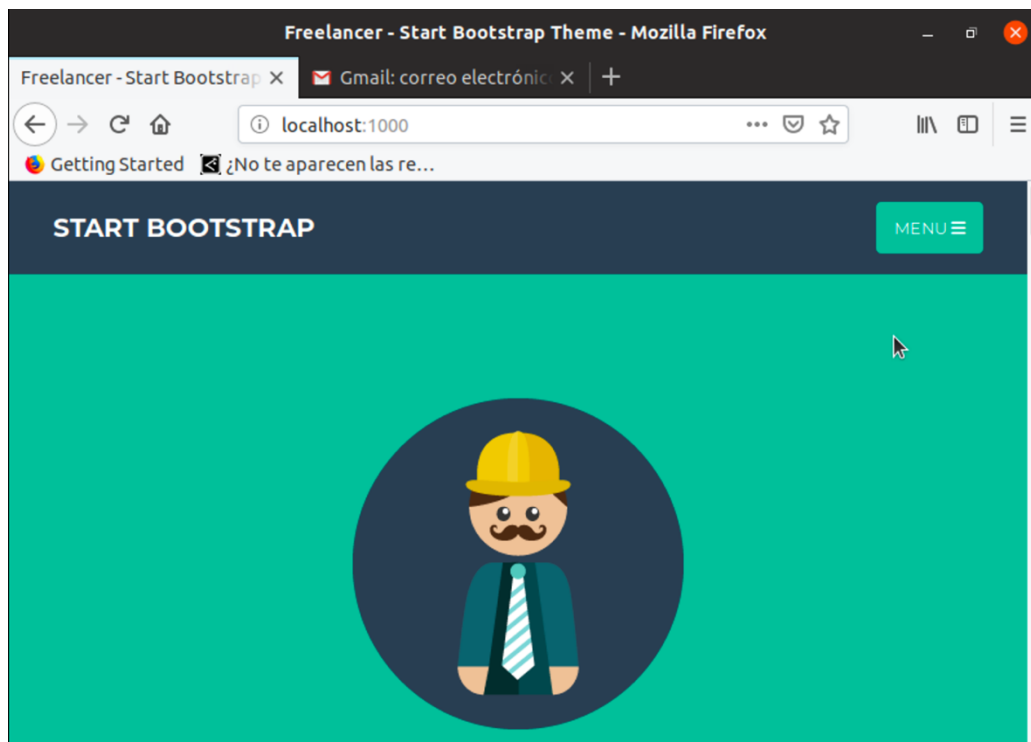
Ejecutamos el comando build para que se genere la imagen:

```
andres@andres-Parallels-Virtual-Platform:~/misitios$ docker build -t bootstrap:1.0 /home/andres/misitios/
```

En este ejemplo hemos puesto el *TAG* para remarcar por ejemplo que es la versión 1.0 de la web que hemos “creado”.

Una vez generada la imagen y habiendo comprobado que se ha generado correctamente, pasamos a crear un contenedor:

```
andres@andres-Parallels-Virtual-Platform:~/misitios$ docker run -d -p 1000:80 bootstrap:1.0
6f55bc545bf32dd4f89e9c1358aa9e499a391546fe890fc37f98015e706ae1d4
andres@andres-Parallels-Virtual-Platform:~/misitios$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS
6f55bc545bf3        bootstrap:1.0      "/bin/sh -c '/usr/sb..." 10 seconds ago      Up                  82/tcp, 0.0.0.0:1000->80/tcp
9 seconds
```



IE MÓDULO SISTEMAS INFORMÁTICOS		
Curso/grupo: 1º DAM	Módulo	0441
Fecha:		UT 2
Entrega:		Manejo Docker

Exportar/Importar imágenes de nuestra máquina

Tenemos la posibilidad de exportar e importar imágenes. Pongamos por ejemplo que queremos trasladar una imagen de las que hemos creado a partir de contenedores personalizados.

Para exportar la imagen:

```
$ docker save -o ruta_absoluta/nombre_a_exportar.tar nombre_imagen
```

```
$ docker save -o /home/andres/apache2.tar apache2
```

Observamos que hemos generado un fichero tar (recordamos que este tipo de fichero es usado para almacenar archivos y directorios en un solo archivo, NO para comprimirlos).

Con la imagen, nos llevamos este archivo a la otra máquina donde queremos importar esta imagen y lo alojamos en algún directorio de dicha máquina. Escribimos entonces:

```
$ docker load -i ruta_absoluta/imagen_a_importar.tar
```

Exportar/Importar contenedor de Docker vía archivo TAR

Migrar un contenedor de Docker

1) Exportamos el contenedor a un archivo tarball

```
$ docker export <CONTAINER ID> > /home/export.tar
```

2) Movemos el archivo a una nueva maquina con docker instalado.

3) Importamos en la nueva máquina.

```
$ cat /home/.../export.tar | docker import -- some-name:latest
```

Donde *some-name* es el nombre con el que queramos definir el contenedor que estamos importando.